Deephaven as an Application Framework: Findings and Guidance

Oct 19, 2026 · @Taylor

Purpose and scope

This note records what we learned building and probing Deephaven applications, and recommends where application code should live on our Enterprise plant.

- **Tested hands-on:** Deephaven Community Core 42.6, first in Docker and then as the pip-installed server running natively on Windows, with deephaven.ui 0.42.2, plotly-express 0.20.1, the `@deephaven/*` web packages 1.30–1.31 and `pydeephaven` 42.6.
- **Not tested on our plant:** Enterprise behaviour comes from Deephaven's public Enterprise documentation. Confirm it against our installed version.

Claims carry one of three markers:

Marker	Meaning
Verified	Observed in a run on Community 42.6
Docs	Stated in Deephaven documentation
Unverified	Our understanding; confirm with the vendor or platform admins

Summary

Deephaven is a declarative dataflow engine, so it is an excellent home for table-shaped logic and a poor host for general-purpose stateful services.

- **Runtime model.** A worker runs its script once to declare a graph of tables. An update-graph clock (about 1 s by default) then advances every table in the worker together; a worker can add update graphs, including an event-driven one that your own code steps. **Verified.**
- **Checkpoints come free.** Each cycle is a consistent snapshot across all tables in the worker. `merged_listen` delivers one callback per cycle across many feeds. **Verified.**

- **Two UI routes.** deephaven.ui is the fastest path to live internal apps: all Python, rendered on the server. React plus the `@deephaven/*` packages gives full control of the frontend, at the cost of setup code and several integration fixes. **Verified.**
- **Python in the engine is the expensive path.** Our two Python callbacks were the costliest nodes in the graph, at about 195 ms per interval each. The costliest Java operation, a rolling update_by, took 80 ms; most took 3–36 ms. **Verified.**
- **Lifecycle bugs fail silently.** A listener that throws stops for good, re-run setup code stacks callbacks, and leaked background threads survive until the worker restarts. **Verified.**
- **Placement.** Derived tables, aggregations, joins and dashboards belong in PQs. Applications with their own servers, lifecycle and dependencies should run as co-located clients that keep heavy table work on the server.
- **Next steps.** Adopt the placement rule, build a pre-approved "paved road" for client apps, and get written vendor answers to the open questions.

How the Deephaven runtime works

A worker runs your script once to declare a table graph; after that, the engine's update-graph clock keeps the graph current, one cycle at a time.

Process lifecycle

This order is visible in the server log. **Verified.**

1. The JVM starts and loads configuration (`START_OPTS`, `deephaven.prop`).
2. Services are wired: authentication, sessions, object resolvers.
3. CPython is embedded through jpy, giving one global Python session.
4. Plugins register: table types, figures, deephaven.ui, JS plugins.
5. The update graph starts cycling.
6. Application scripts run, top to bottom, and must return.
7. The gRPC/HTTP port opens and clients can connect.
8. Steady state: update cycles plus client requests, until shutdown.

Three consequences follow:

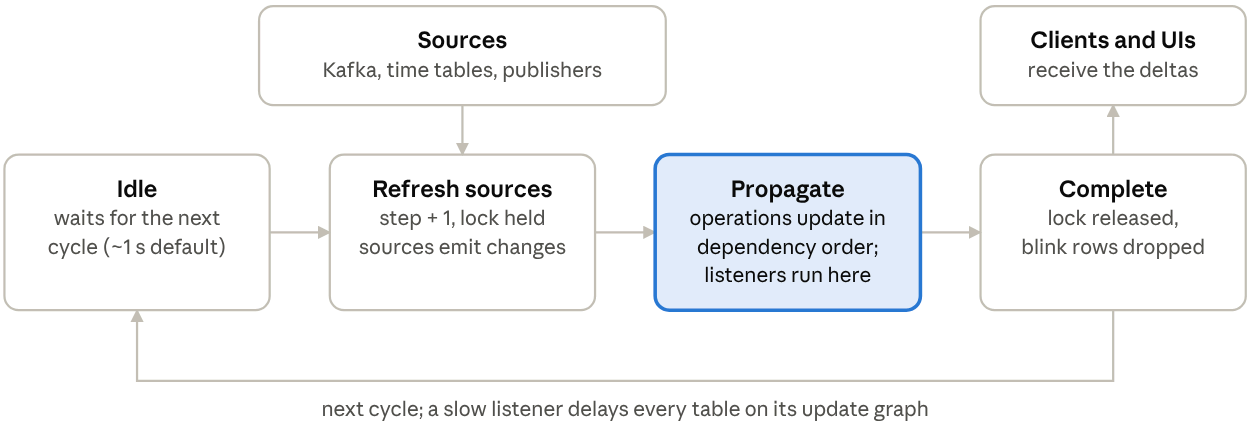
- **The script is your only initialize hook.** A ticking source starts ticking as soon as it is created; there is no separate "start feeds" phase.

- **Nothing is restored on restart.** The engine does not checkpoint a worker's in-memory state; rebuild state from tables in the startup script.
- **Shutdown is Java-side.** The JVM has an ordered shutdown manager, but we found no supported Python shutdown hook. Treat shutdown as abrupt. **Unverified.**

The update-graph cycle

Each cycle refreshes every source, propagates the changes through the graph in dependency order, then publishes the result. **Verified.**

Each cycle refreshes sources, propagates changes and publishes one step



update-graph cycle · four states, repeating

Your code runs in the highlighted step: a listener's time is added to every cycle for every table in the worker.

- **A cycle is a consistent checkpoint.** Every table in the worker reflects the same step, and a listener reading another table inside its callback sees that same step.
- **There is one default clock per worker.** All sources and listeners on it share the cadence, and rows that arrive mid-cycle wait for the next refresh. A worker can also run extra update graphs, including an event-driven one that advances only when your code calls requestRefresh() (Verified).
- **Updates are incremental.** Operations receive row-level changes (added, removed, modified, shifted) and update their results in place.
- **Listeners run inside propagation.** A slow Python listener delays the cycle for every table in the worker.

Threads

Thread	Does	Interaction with the cycle
Update-graph refresh and update threads	Refresh sources, propagate changes, run listeners	Hold the exclusive lock while updating
Client request threads	Fetch tables, set viewports, create operations, write input tables	Take consistent snapshots without blocking cycles; input-table writes land in a later cycle
Script session (serial)	Runs console and `runCode` / `run_script` commands in the global scope	One command at a time
Server executors (`serial`, `concurrent`)	Python work submitted with `submit_task`	Run on JVM threads; use `serial` for anything taking the exclusive lock
Your own threads	Feed adapters, timers, an asyncio loop	Need an execution context captured on a thread that holds it; push rows through `table_publisher`; must not change the graph outside the lock rules

All Python on every thread shares one interpreter lock (the GIL).

Liveness and long-running work

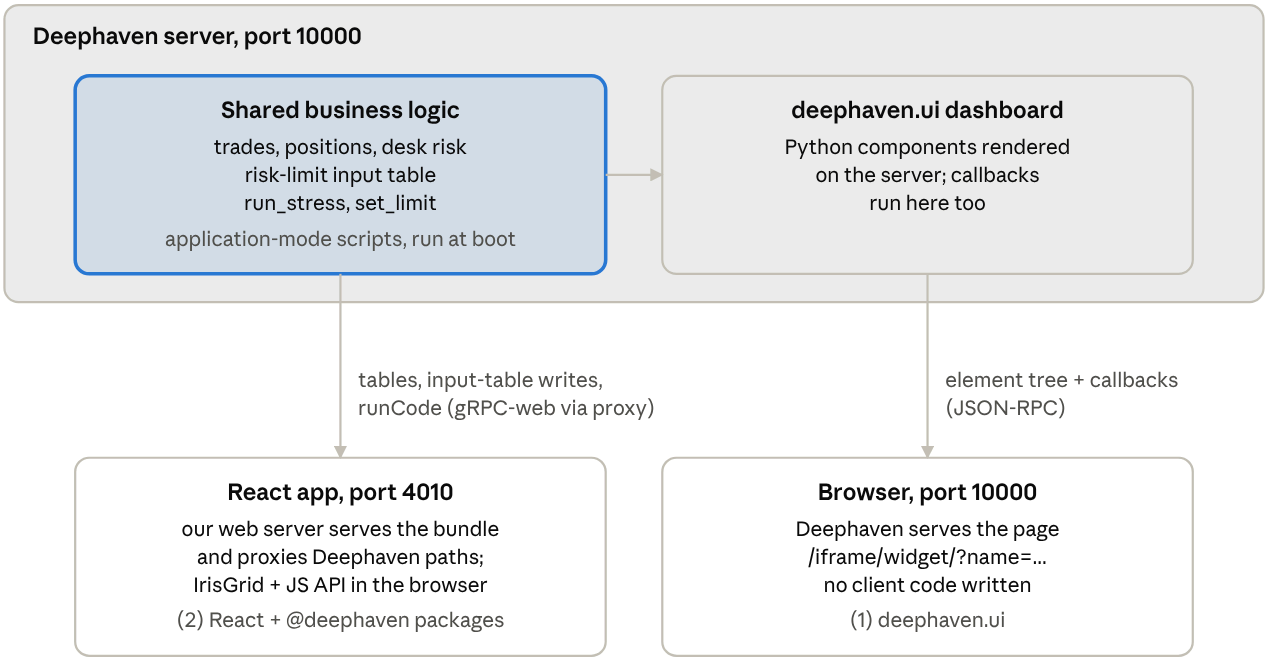
- **References keep tables alive.** A ticking table updates only while something references it, and a name in the global scope is such a reference. In Community 42.6, top-level names in application-mode scripts already live in the global scope, so they need no `global`.
- **Do not loop in a script.** A startup script that never returns blocks the worker's startup.
- **Sources do the waiting.** Use `time_table`, the Kafka consumer or remote-table subscriptions; for a custom feed, run your own thread and push rows into a `table_publisher`.
- **Prefer query-language formulas.** They compile to Java; Python called per row inside `update` takes the GIL on the update thread.

Building UIs: deephaven.ui vs React + web-client-ui

Both routes work against the same live tables: deephaven.ui trades control for speed, and React with the `@deephaven/*` packages trades speed for control.

We built the same market-making book monitor both ways on one Community server. A risk limit written from the React app showed up as a breach in the deephaven.ui dashboard within a tick. **Verified.**

Both UI routes read the same live tables on one Deephaven server



A limit written from the React app appeared as a breach in the deephaven.ui dashboard within a tick.

test architecture · one server, two UI routes

The business logic runs once in the server; only where the UI code runs, and who serves the page, differs between the routes.

	deephaven.ui	React + `@deephaven/*` packages
Code written	About 190 lines of Python	About 350 lines of TypeScript, plus build config
Served by	The Deephaven server itself	Our own web server (Vite), proxying to Deephaven

	deephaven.ui	React + `@deephaven/*` packages
Where UI logic runs	On the server, in Python	In the browser, in React
Live cards	`ui.use_row_data(table)`	Viewport subscription plus an update listener
Grids	`ui.table` with server-side conditional formats and data bars	`IrisGrid`: the same grid, but no declarative formatting API
Writing to the server	Plain Python call	`table.inputTable()` then `addRow()`, no server code
Calling server logic	`ui.use_memo` around a Python function	`runCode` into the shared global scope, then fetch the result
Per-user state	Automatic, per session	Ours to build (we suffixed variables per browser tab)
Charts and layout	`dx.line(...)` in one line; dockable dashboards included	Extra packages and wiring

When to choose which

- **deephaven.ui** suits internal and analyst tools written by Python teams. Costs: every interaction is a server round trip, rendering uses server CPU per connected user, and the page shell belongs to Deephaven.
- **React + `@deephaven/*`** suits the few apps that need their own routing, design system, other data sources or a separate release cycle. Costs: a bootstrap chain to wire, table and session lifecycles to manage, and packages that are built for Deephaven's own monorepo first.

Integration notes for the React route

- **Serve from one origin.** Proxy `/jsapi`, `/js-plugins`, `/arrow.*` and `/io.deephaven.*` (with websockets) to the Deephaven server to avoid CORS. **Verified.**
- **Bootstrap chain.** `ApiBootstrap` loads `dh-core.js` from the server; `AppBootstrap` then wires the client, authentication and connection. **Verified.**
- **Pin `@deephaven/jsapi-types` to ~41.11.** The 1.3x UI packages depend on it; installing 42.x creates duplicate, incompatible types. The runtime API always comes from the server. **Verified.**

- **Shim `@paciolan/remote-module-loader`.** `@deephaven/app-utils` default-imports this CommonJS module and fails under Vite with "is not a function". **Verified.**
- **`IrisGrid` takes no `dh` prop.** The package README still shows one; `dh` now comes from context. **Verified.**
- **Calling Python via `runCode` shares one global scope.** Concurrent users can overwrite each other's variables, and nothing cleans them up. A custom object-type plugin with a message stream, which is how deephaven.ui works, is the robust alternative. **Verified** (scope), **Unverified** (plugin effort).
- **On Enterprise** the client loads an Enterprise-specific API, logs in through Enterprise authentication and connects to a chosen PQ; our local proxy setup does not carry over unchanged. **Unverified.**

Deephaven as an application framework

Much of a hook-based event framework already exists in Deephaven, but the engine owns time and the graph, so embedding such a framework in a worker means two runtimes in one process.

The framework we compared against is a typical trading-style design. Users subclass an `Algorithm` and override hooks (`on_initialize`, `on_data`, `on_order_update`, `on_start_of_day`, `on_shutdown` and so on). An `EventManager` owns an asyncio loop, the feed tasks, a synchronizer that flushes feeds at checkpoints, a processing DAG, and side servers: a websocket API, a diagnostics site and a REPL port.

The core difference: an event framework pushes events to user code and lets user code change the plan at any time. Deephaven runs a graph that user code declared up front, and user code is mostly gone after setup.

How the hooks map

Framework piece	Deephaven equivalent	Fit
`on_initialize`	The startup script body	Native
Kafka feeds	`kafka.consumer.consume(...)` returns a ticking table	Native
Feeds from other plants/PQs	`uri.resolve("pq://...")` or `remote_table`	Native (Docs)

Framework piece	Deephaven equivalent	Fit
Solace, kdb, custom feeds	Your own thread pushing into a `table_publisher`	Workable. Enterprise has Solace streaming import (Docs); we know of no kdb connector
Synchronizer checkpoints	The update-graph cycle	Native: the default cycle, or an event-driven graph your code steps (Verified)
Processing DAG	Table operations, updated incrementally	Native, and better than hand-rolled for table-shaped data
`on_data`, `on_order_update`, `on_signal_update`	Listeners: `listen`, or `merged_listen` across several tables	Workable, with constraints below
Sinks	`kafka.producer.produce(...)`, or listeners writing out	Native / workable
Scheduled callbacks, market open/close	`time_table` plus a listener, or your own timer thread	Workable
`on_start_of_day` / `on_end_of_day`	In Enterprise, the PQ schedule: the worker starts and stops	Different model: a fresh worker each day
Historical catch-up	`db.historical_table` + `db.live_table` (Enterprise); `TableReplayer` (Core)	Native (Docs)
`on_new_subscriptions`, `on_new_datafeed`	Creating tables at runtime	Awkward: constrained inside callbacks (Unverified detail)
`on_deserialized`	None	Rebuild state from tables at startup

Framework piece	Deephaven equivalent	Fit
`on_shutdown`	No reliable Python hook found	Treat shutdown as abrupt (Unverified)
Websocket API for JS clients	JS API, deephaven.ui, or a custom object-type plugin	Native, but Deephaven's protocol, not ours
Diagnostics dashboard	deephaven.ui plus the perfmon tables	Native
REPL into live state	The console (Community); attaching to a running PQ is unclear	Unverified for Enterprise

Where the runtime pushes back

- **The default checkpoint is global.** The default clock is shared by everything in the worker, with a latency floor of about one cycle plus listener time. An event-driven graph lifts this: tables built on it advance only on requestRefresh(), and in our test one refresh delivered three pushes as a single checkpoint (Verified).
- **Callbacks run on the update thread.** A slow callback stalls every table on its update graph (by default, every table in the worker), so real work has to be handed to another thread, which reintroduces a second runtime; the server's executors (submit_task) can run it on JVM threads instead (Verified).
- **Changing the graph from a callback is constrained.** The usual pattern defers creation to another thread under the update-graph lock (`deephaven.update_graph`). **Unverified detail.**
- **Side servers do not fit.** Inbound ports, TLS and routing are not provided for workers. A REPL port inside a worker would also sit outside Deephaven's authentication, permissions and audit.

Three realistic shapes

1. **Deephaven-native rewrite.** The framework becomes a thin library of conventions on Deephaven primitives. Good when algorithms are table-shaped; an event-driven update graph removes the fixed one-second checkpoint.
2. **Standalone framework, Deephaven as a subprocess or remote service.** Keep the asyncio runtime, its checkpoint semantics and its servers, and reach Deephaven only as a client. **Verified** end to end (`tests/host_service_demo.py`).

3. **Embedded service on a thread.** Start the pip server on a background thread of our own process. We get in-process tables and listeners with no network hop, and our synchronizer can step an event-driven graph directly. The costs are a shared GIL (listener lag doubled under host CPU load), our `__main__` becoming the console scope that clients can read and modify, one server per process, and per-thread execution contexts. **Verified** (`tests/embedded_thread_demo.py`, `tests/probe_event_driven_clock.py`). Community only; Enterprise PQ workers are not embeddable.
4. **Hybrid (recommended for the plant).** Heavy, shared, table-shaped transforms run in PQs; the framework runs standalone and subscribes to their results.

Experiments and observed results

Four small experiments on Community 42.6 confirmed the cycle model and put the latency floor for externally pushed rows at about one second, the default cycle length. All results here are **Verified**.

Experiment	What it does	What we observed
1. Lifecycle	A script creates a 0.5 s `time_table` and returns	The script finished in milliseconds; ticks kept arriving. Pairs of ticks shared one `Step` and nearly one timestamp: one engine cycle per second processed both
2. Declarative DAG	Bars, VWAP, a 50-tick rolling average and a long/short signal, declared once	Signals flipped between snapshots with no user code running per tick
3. Listener as `on_data`	A Python listener detects signal flips from modified rows and their previous values, updates Python state and writes to a sink table	About one callback per cycle; 9 flips counted produced exactly 9 sink rows, each tagged with its cycle

Experiment	What it does	What we observed
4. Feeds and checkpoints	Two asyncio tasks push orders and signals via `table_publisher`; one engine-native quote feed; one `merged_listen` synchronizer	One callback per cycle with about 4 quotes, 4–5 orders and 1–2 signals together. A table read inside the callback always matched the step. Pushed rows waited 650–1,000 ms before a cycle picked them up

What this means for design:

- **The checkpoint you get is the engine's.** It is consistent and free, but its cadence is set per worker, not per algorithm.
- **Latency is bounded below by the cycle.** The cycle length is configurable, but shortening it applies to everything in the worker.
- **Incremental computation is real.** Derived tables did their work without any per-tick Python, and they were not the bottleneck.

Gotchas and lifecycle pitfalls

Most failures we hit were silent lifecycle bugs. They matter more inside a long-lived, shared worker than in a standalone process. All entries are **Verified** on Community 42.6.

Engine and listeners

Gotcha	Symptom	Fix
An exception in a listener stops it permanently	One traceback in the log, then no more callbacks; feeds keep flowing and the worker looks healthy	Wrap every hook in `try/except` and record failures somewhere visible, such as a status table
`merged_listen` passes every watched table each call	`None` for tables that did not change that cycle; code that assumes an update crashes	Skip `None` entries

Gotcha	Symptom	Fix
Re-running setup code stacks callbacks	Outputs written twice; rebinding the handle variable does not stop the old listener	Call `handle.stop()` on the previous handle before registering a new one
Background tasks that read lifecycle state from globals leak	Old asyncio loops saw the next run's unset stop event and ran forever; only a worker restart cleared them	Pass stop events and other lifecycle objects into tasks as arguments
Execution contexts are per thread	On a thread without the real context, `get_exec_ctx()` returns a static-only one and creating a ticking table fails with `UpdateGraphConflictException`	Capture the context where it is real (inside `run_script`, or on the thread that started the server) and enter it with `with ctx:` on your thread
Tables matched by `toString()` collide	Updates labelled as the wrong table	Match tables by identity (`j_table.equals`)
Formulas recompute only when their input columns change	A formula reading no columns, on a `last_by()` row, never refreshed	Append a new row per refresh, or reference a changing column
Underscore names are published	`_private` tables still appear as `scope` fields	Keep private tables out of the global scope: locals, an object attribute, or `del`
Docs advise `global` in application-mode scripts	In Community 42.6 the scripts already run in the global scope; undeclared top-level names become fields	Not needed here; keep it only where a version or Enterprise setup is confirmed to need it
A shared ticket dies with its publisher	Fetching it, and copies fetched while it was valid, fail once the publishing session closes	Use `bind_table` for a durable, named handle
One server per process	A second `Server(...).start()` fails: "Cannot create more than one instance of the server"	A restart means a new process

Gotcha	Symptom	Fix
`use_row_data` returns pandas-typed values	Long columns arrive as floats (`514.0`)	Cast with `int()`

Tooling and packages

Gotcha	Symptom	Fix
`@deephaven/jsapi-types` 42.x with 1.3x UI packages	Duplicate, incompatible TypeScript types	Pin `~41.11`
`@paciolan/remote-module-loader` interop	`TypeError: ... is not a function` at startup under Vite	Alias it to a small shim that normalizes the default export
Stale `IrisGrid` README	`<IrisGrid dh={dh}>` fails type-check	Use `<IrisGrid model={model} />`
`pydeephaven` 42.6 with `bitstring` 5	`TypeError` creating a `Session`	Pin `bitstring<5`
pip server on native Windows (documented as WSL-only)	JVM crash in `jimage.dll`, then "Invalid path: /layouts"	Put the JVM's `bin` and `bin/server` folders on `PATH` and make the web storage directories relative; `server/run_server.py` does both
Ancestors log opened late	Graphs show a node but none of its ancestors	Open `update_performance_ancestors_log()` at worker start
Stock image lacks Graphviz	`ancestor_svg` / `ancestor_image` fail: "None of the provided engines could be initialized"	Add the `graphviz` package to the server image

Introspection and profiling

Deephaven ships a per-node profiler and a dependency-graph renderer. With two setup changes, they show every table operation and listener in a worker with its cost.

Tool (`deephaven.perfmon`)	Shows	Notes
`update_performance_log()`	Per node, per interval (about 1 min): usage and CPU time, invocations, rows added/modified/removed, allocations	Every table operation and every listener is a node. **Verified**
`update_performance_ancestors_log()`	Each node's upstream nodes	Only records nodes created after it is first opened. **Verified**
`ancestor_dot` / `ancestor_svg` / `ancestor_image`	The graph upstream of a node, as Graphviz text, SVG, or a deephaven.ui panel	SVG and image need the Graphviz binary in the image. **Verified**
`server_state_log()`	Cycle times, memory, garbage collection	**Verified** (available)
`query_performance_log()` , `query_operation_performance_log()` and tree-table variants	How long building each table took	**Verified** (available)

What it showed

- **Python callbacks were the costliest nodes**: about 195 ms per interval for each of our two listeners, against 80 ms for the rolling update_by and 3–36 ms for the other Java table operations. **Verified.**
- **The engine's own structure is visible**, for example that `update_by` partitions by key and validates each partition internally.

Image — Update graph of experiments 2–4, drawn from the engine's own performance logs

The engine's graph for experiments 2–4, drawn by our readable renderer: green nodes are sources, blue are table operations, orange are Python callbacks, each with its cost per interval.

Setup requirements

- Open the ancestors log at worker start, before any tables are built. On Community we did this from the first application-mode script.
- Add Graphviz to the server image for SVG output. Text (DOT) output works without it.
- The raw labels are Java descriptions. We wrote a small renderer (`examples/pretty_graph.py`) that adds Python variable names, plain-language operations, per-interval cost and downstream arrows, and hides engine-internal nodes.

Blind spots

- **Only engine nodes appear.** Asyncio tasks, Python state and queues are invisible; a `table_publisher` feed shows only as its output table.
- **Graphs are per node.** They show what feeds one node, not a whole-worker map, and there is no live view of a cycle in progress.
- **On Enterprise** these logs are, we believe, persisted per worker as queryable tables, so past PQs can be examined. **Unverified.**

Other tools worth trying: the server appears to support attaching a Python debugger (a debugging initializer runs on the script thread), and standard JVM tools such as jstack and Flight Recorder apply. **Unverified.**

Enterprise specifics

Each PQ is roughly one Core engine in its own worker, started and stopped on a schedule by the controller; PQs share tables by network subscription, not shared memory.

The PQ model

- **One worker per PQ.** Each PQ runs in its own worker JVM with its own heap and Python interpreter; Core+ workers run the same engine as Community. **Unverified** (Docs list worker and heap configuration).
- **A PQ is code plus settings.** It holds Python or Groovy code and settings such as query type, required heap and who can access it. **Docs.**
- **PQ types.** Live Query (Script), Live Query (ReplayScript) and Batch Query (RunAndDone) run on query servers. Merge, import and validation types run on merge servers and need elevated privileges. **Docs.**
- **Interactive consoles are temporary workers** of the same kind, without a schedule. **Docs** (listed as an internal PQ type).

	Community (what we tested)	Enterprise
Start and stop	You start the server; scripts run at boot	Controller starts and stops workers on the PQ schedule; a dispatcher launches each worker (Docs)
Scale	One server	Many PQs across query and merge servers (Docs)
Authentication	Anonymous or pre-shared key	Central authentication, per-PQ permissions, row and column access rules (Docs)
Data	Generated or ingested in-script	Shared database: `db.live_table`, `db.historical_table` (Docs)
Web UI	Served by the same process	One shared web UI connecting to whichever PQ owns a widget (Unverified)

Sharing tables between PQs

Both methods copy data into the consuming worker and keep it ticking; the producer needs no extra setup. **Docs.**

```
from deephaven import uri
positions = uri.resolve("pq://RiskEngine/scope/positions")

from deephaven_enterprise import remote_table
positions = remote_table.in_local_cluster(query_name="RiskEngine",
table_name="positions").subscribe()
```

URI options cover static snapshots, column selection, blink mode, reconnect behaviour and an initial-wait timeout. `remote_table` can also filter on the producer and reach other clusters. Splitting work across PQs therefore already pays the cost of a network copy.

Running non-Deephaven applications inside a PQ

It works mechanically if the event loop runs on a background thread, but it is a worse deployment target than a container. A top-level `asyncio.run()` never returns, so the PQ never finishes starting.

- **Shared interpreter.** The app competes for the GIL with every Python formula, listener and UI render in that worker.
- **Memory outside the heap limit.** Python memory is not counted against the PQ's configured heap, so a leak can exhaust the whole query server. **Unverified** in Enterprise accounting.
- **Lifecycle mismatch.** Scheduled stops are hard stops, failover can run two copies at once, and asyncio signal handlers work only on the main thread.
- **No application health.** A dead background thread leaves the PQ showing as running, and nothing restarts it.
- **No ingress.** No assigned ports, TLS or routing for inbound HTTP or websocket traffic.
- **Shared dependencies.** Workers on a server typically share an admin-managed Python environment. **Unverified.**
- **Identity.** The app runs with the worker's account, credentials and data access, outside the platform's audit of what it does.

Deephaven's marketing promises fast "analyses, models, algos, reports, and dashboards" and tooling for "launching, scheduling, and monitoring applications". It does not claim that arbitrary services run well as PQs, nor does it describe resource isolation between workloads. **Docs** (Enterprise product page).

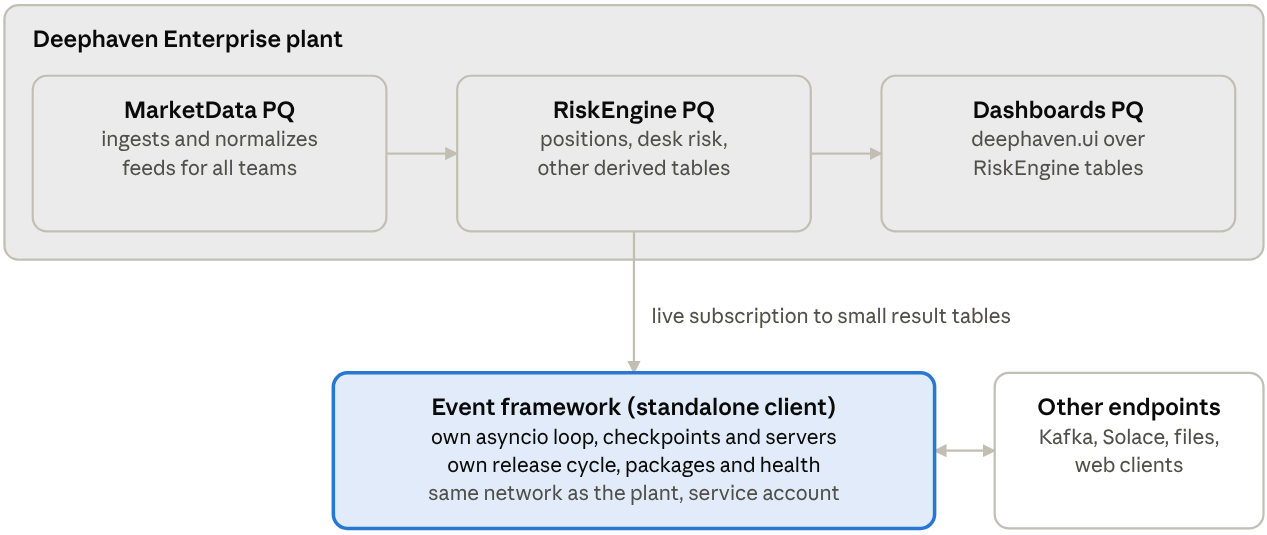
Placement guidance: PQ or standalone client

Put table-shaped work in PQs; run stateful applications that have their own servers as co-located clients that keep heavy table work on the server.

The one-question test: **is the app mostly table operations, or does it mostly do other things with some table data?**

Workload	Where	Why
Derived and real-time tables, analytics, risk calculations, aggregations	PQ	Table-heavy work is what the engine is built for
Dashboards over Deephaven data	PQ (deephaven.ui)	Built in; per-user state for free
Thin feed handlers, alerts on table changes	PQ, with guardrails	Small and table-centric. Add a heartbeat or status table and keep per-message Python light
Apps with their own users or APIs, workflow state, external integrations, heavy non-table compute, independent release or scaling, or long dependency lists	Standalone client, co-located	Needs service lifecycle and isolation; locality is kept by running table work server-side

Hybrid: table work in PQs, the event framework as a co-located client



example hybrid layout · three PQs and one client app

Shared, table-shaped work stays in PQs that other PQs and apps subscribe to; the stateful framework keeps its own runtime and pulls only the results it acts on.

Weighing the arguments for "everything as a PQ"

- **Data locality is real for in-worker table operations.** But PQ-to-PQ sharing is already a network copy, and a client can create tables on the server and subscribe only to small results. Placed in the same network, a client keeps most of the benefit.
- **Deployment and SDLC savings are real.** But a PQ script is code. Deploying an application as "configuration" puts it under a lighter change-control regime, which tends to draw audit scrutiny once something important runs that way.
- **The savings move cost rather than removing it.** All-in means one failure domain, one shared Python environment, and every Deephaven upgrade becoming a regression test for every app on it.
- **Testing is weaker.** Unit tests, CI, code review and rollback are harder for PQ scripts than for a normal repository and pipeline. Enterprise supports Git-sourced PQ scripts (Docs), which narrows this gap; confirm how our installation uses it. **Unverified.**

Recommended next steps

- [ ] Agree the placement rule above as the default, with exceptions argued case by case.

- [ ] Build a paved road for client apps: a container template, a CI pipeline, and a standard service-account connection, approved once by SDLC so each new client app costs about what a PQ does.
- [ ] Run a measured pilot: one real, complex app built both ways, comparing latency, throughput, memory and operational risk.
- [ ] Add guardrails for PQs that do host app-like code: a dedicated query server, wrapped hooks, a status table, and git-backed scripts.
- [ ] Send the vendor questions below and record the answers here.

Open questions for the vendor

These questions turn the Unverified items into written answers; ask the Deephaven account or solutions team for responses on our installed version.

1. **Resource isolation.** If one PQ's Python leaks memory outside the JVM heap, what protects other PQs on that query server? Are there per-worker memory or CPU limits?
2. **Dependencies.** Can one PQ have its own Python environment, or are environments shared per server? What is the supported process for adding or upgrading a package?
3. **Long-running non-table code.** Is an asyncio service on a background thread inside a worker a supported pattern? What happens to it on scheduled stop, failover and restart?
4. **Inbound traffic.** Is a PQ hosting an HTTP or gRPC API supported? If so, how are ports, TLS and routing handled?
5. **Health.** Can the controller detect that application code inside a running PQ has failed, and restart it?
6. **Graph changes at runtime.** What are the exact rules for creating or closing ticking tables from inside a listener?
7. **Shutdown.** Is there a supported Python hook that runs on scheduled stop?
8. **Console attach.** Can an authorized user open a console in a running PQ's scope?
9. **Cycle tuning.** Can the update-graph cycle length be set per PQ, and are additional or event-driven update graphs supported inside PQ workers?
10. **Upgrades.** When Deephaven is upgraded, what regression burden should PQ owners expect?
11. **Reference customers.** Do customers run general-purpose services, rather than analytics, as PQs in production? What do those look like?

Reference

Everything above can be reproduced from the local project `deephaven-compare`, which needs Docker Desktop or a Java 17+ runtime for the pip server, plus Node 18 or later for the React client. It is not yet in version control.

Path	What it is
`VERIFIED_FACTS.md`	The ledger of every verified fact, with the test behind it
`docker-compose.yml`, `server/Dockerfile`	Deephaven 42.6 plus Graphviz, application mode, anonymous authentication (localhost only)
`server/run_server.py` (+ `setup_wsl.sh`, `start.sh`)	The same server as a plain Python process via pip `deephaven-server`; runs natively on Windows
`server/app.d/introspection.py`	Opens the performance logs at boot so graph ancestry is complete
`server/app.d/business_logic.py`	Mock market-making book: trades, positions, desk risk, risk-limit input table, `run_stress` and `set_limit`
`server/app.d/ui_dashboard.py`	The deephaven.ui version of the app
`react-client/`	The React + `@deephaven/*` version, served by Vite on port 4010
`examples/01` – `05`	The experiment ladder and introspection
`examples/pretty_graph.py`, `dump_pretty.py`, `run.py`	Readable graph renderer, and the script driver
`introspect/`	`dh_introspect.py` (runs inside a worker) and its client
`tests/`	The probes behind the Verified marks: scopes and fields, introspection, runtime features, event-driven clock, embedded thread, subprocess hosting

To run it:

```
# Option A: Docker
docker compose up -d --build
# Option B: the pip server, no Docker (Java 17+ in JAVA_HOME)
py -3.12 -m venv .venv-server; .\.venv-server\Scripts\pip install -r
server\requirements-server.txt
.\.venv-server\Scripts\python server\run_server.py
# React client, and the experiment driver
cd react-client; npm install; npm run dev
.\.venv\Scripts\python examples\run.py examples\01_lifecycle.py --show
ex1_ticks --every 2 --times 3
```

The deephaven.ui app is at `http://localhost:10000/iframe/widget/?name=book_dashboard` and the React app at `http://localhost:4010`.

Sources

- deephaven.ui documentation and its architecture page
- Use the JS API
- Application mode
- Embed with iframes
- Anonymous authentication
- Docker install
- JS API `InputTable` reference
- Enterprise PQ overview
- Enterprise remote tables and URIs (Python)
- Deephaven Enterprise product page
- web-client-ui repository (the `embed-widget` package was the reference for the React bootstrap)